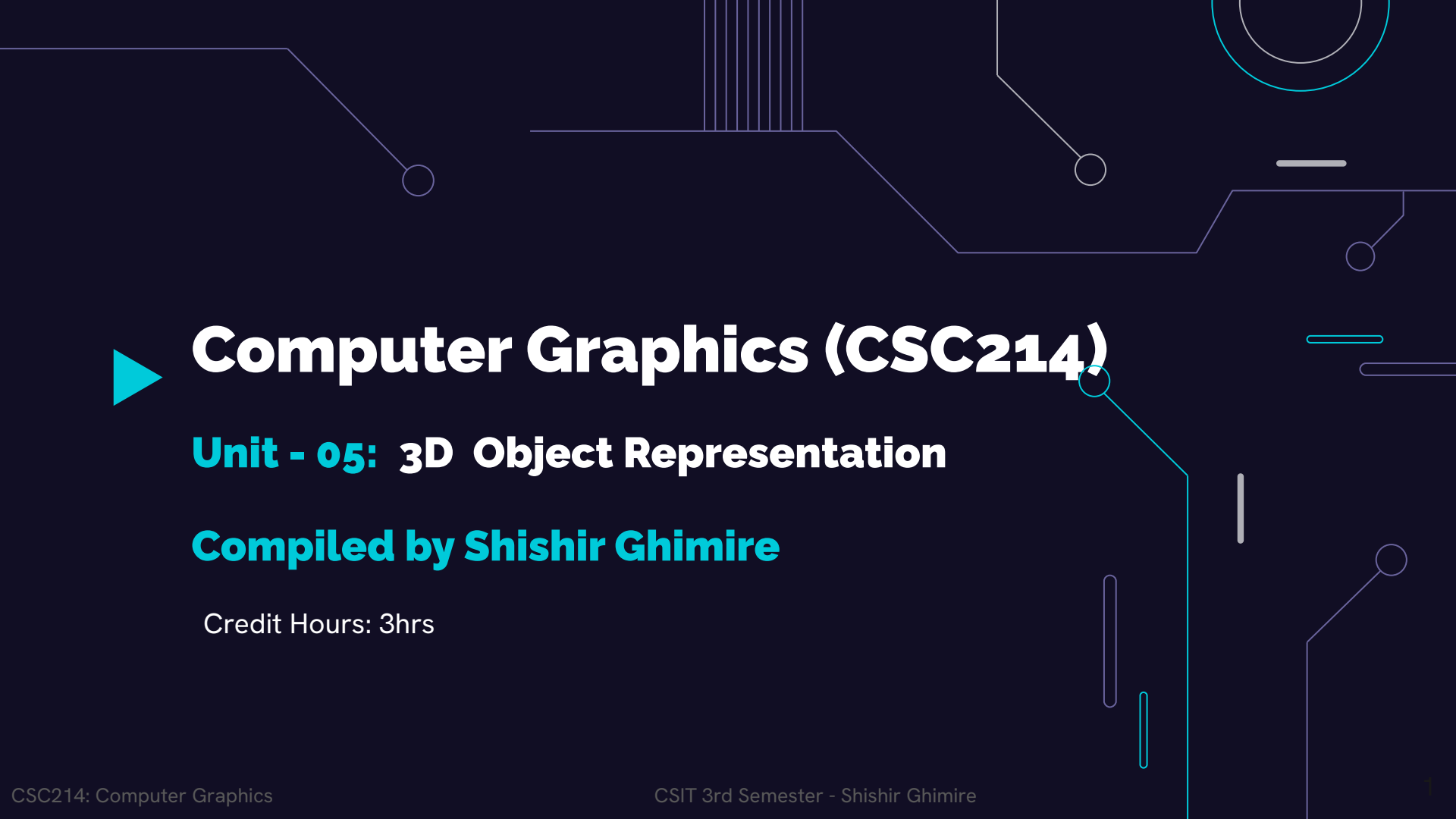




► Computer Graphics (CSC214)

Unit - 05: 3D Object Representation

Compiled by Shishir Ghimire

Credit Hours: 3hrs

► TABLE OF CONTENTS

Unit 5: 3D Objects Representation (7 Hrs.)

5.1 Representing Surfaces: Boundary and Space partitioning

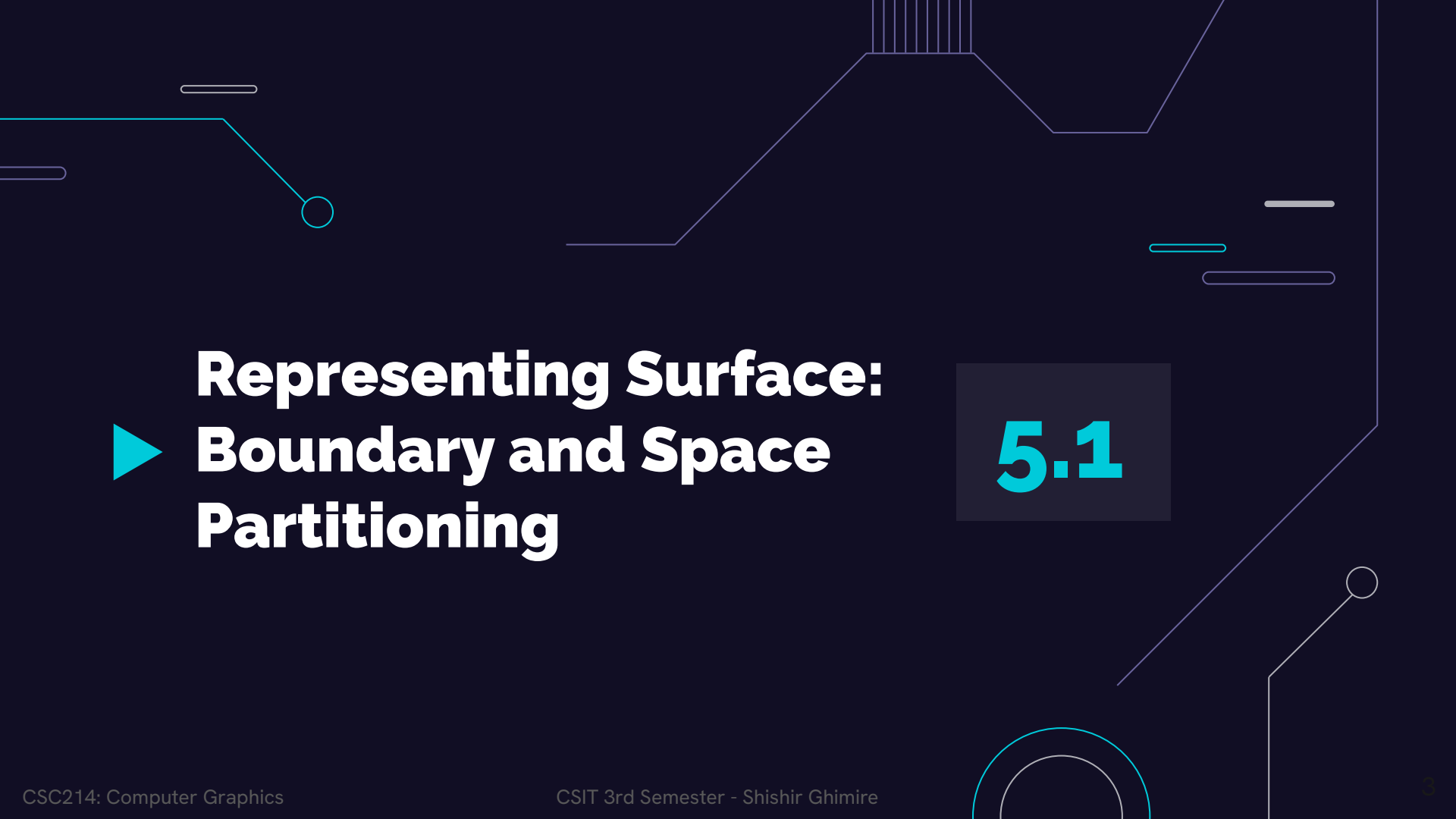
5.1.1 Polygon Surface: Polygon tables , Surface normal and Spatial orientation of surfaces, Plane equations, Polygon meshes

5.1.2 Wireframe Representation

5.1.3 Blobby Objects

5.2 Representing Curves: Parametric Cubic Curves, Spline Representation, Cubic spline interpolation, Hermite Curves, Bezier and B-spline Curve and surface

5.3 Quadric Surface: Sphere and Ellipsoid



Representing Surface: ▶ Boundary and Space Partitioning

5.1

► 3D Object Representation:

3D object representation is divided into two categories:

❖ Boundary Representations (B-reps):

- The boundary representations describe a three dimensional object as a **set of surface** that **separates the object interior** from the environment. E.g. Polyhedron, ellipsoid, aircraft, medical images etc.
- B-reps for single polyhedron (3D object with flat faces) satisfy Euler's formula: $V-E+F=2$

❖ Space Partitioning Representation:

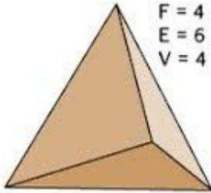
- Space partitioning **describes interior properties** by partitioning the spatial region containing an object into a **small, non- overlapping, contiguous solids**.
- E.g. 3D object as Octree representation.

► 3D Object Representation:

Euler's Polyhedron Formula

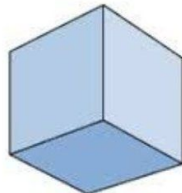
$$V + F - E = 2$$

here, F = number of faces,
E = number of edges,
V = number of vertices



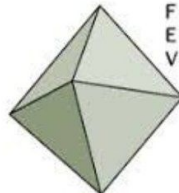
F = 4
E = 6
V = 4

$$4 + 4 - 6 = 2$$



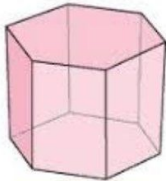
F = 6
E = 12
V = 8

$$8 + 6 - 12 = 2$$



F = 8
E = 12
V = 6

$$6 + 8 - 12 = 2$$



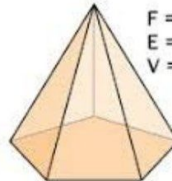
F = 8
E = 18
V = 12

$$12 + 8 - 18 = 2$$



F = 20
E = 30
V = 12

$$12 + 20 - 30 = 2$$



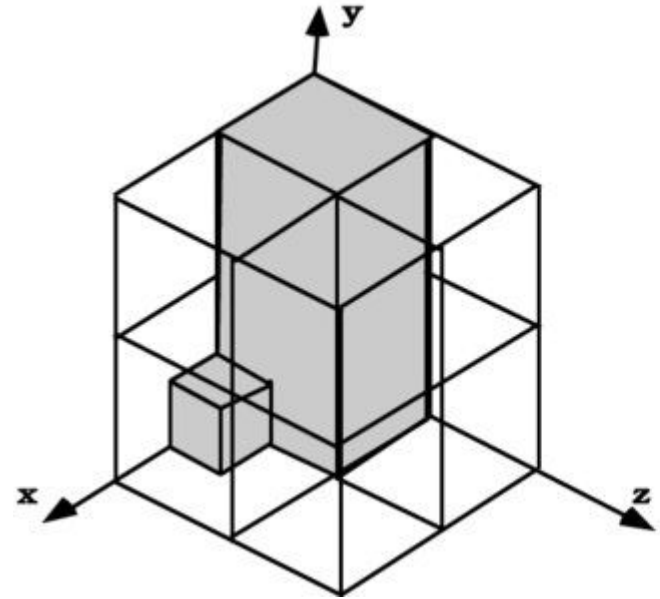
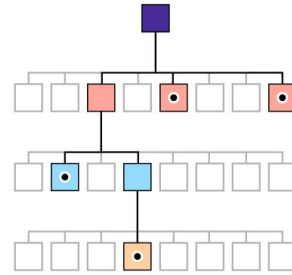
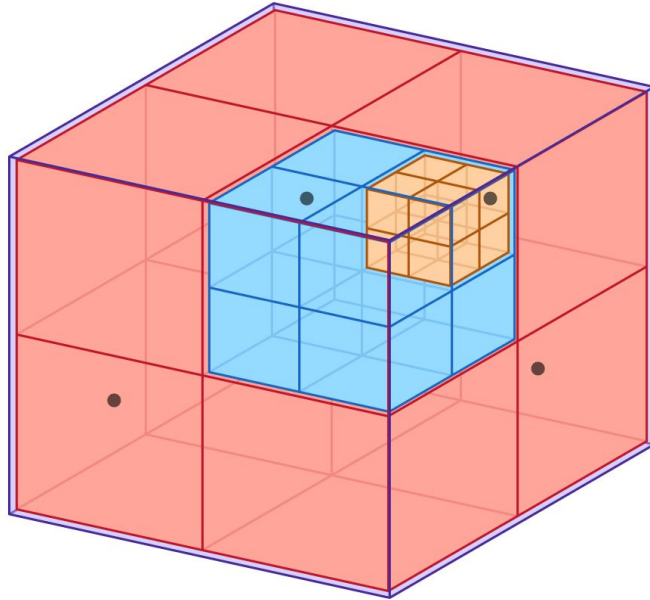
F = 6
E = 10
V = 6

$$6 + 6 - 10 = 2$$



Leonhard Euler

► 3D Object Representation ► **Octree** :

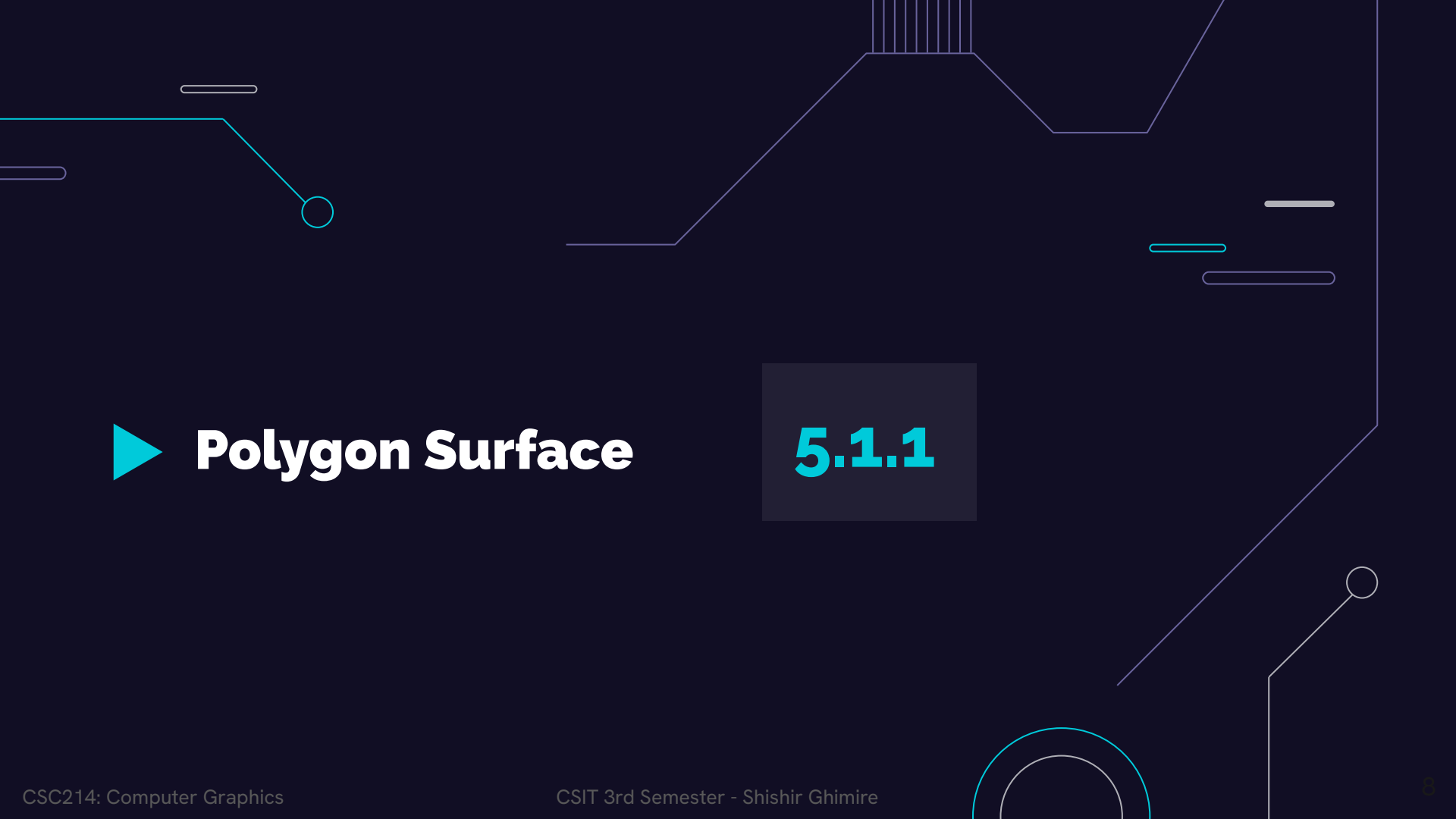


► Boundary Representation:

1. Polygon Surface
 - a. Polygon Table
 - i. Geometric Table
 - ii. Attribute Table
 - b. Polygon Table using Forward pointer in edge table
 - c. Polygon Meshes
 - d. Plane Equations
2. Quadratic Surface
 - a. Sphere
 - b. Ellipsoid
3. Wireframe Representation
4. Blobby Effect
5. Spline Representation
6. Cubic Spline
7. Bezier Curve
8. B-Spline Curve

Planar Boundary Representation

Non-Planar Boundary Representation



► Polygon Surface

5.1.1

► Boundary Representation:

1. Polygon Surface

- ❖ It is **most common** representation for 3D graphics object.
- ❖ In this representation, a 3D object is represented by a **set of surfaces** that enclose the object **interior**. This method simplifies and speeds up the surface rendering and display of the object.
- ❖ The polygon surfaces are **common in design and solid-modeling** applications, since wire frame display can be done quickly to give general indication of surface structure.
- ❖ Polygon Surface is represented by:
 - Polygon Table

► Polygon Surface:

a) Polygon Table

- ❖ In this method, a polygon surface is specified with a **set of vertex coordinates** and associated **attributes**. Polygon data tables can be organized into two groups:
 - **Geometric tables:** It contain vertex coordinates and parameters to identify the spatial orientation of polygon surfaces
 - **Attribute table:** It gives attribute information for an object (Degree of transparency, surface reflectivity etc.).
 - Geometric data consists of **three tables**:
 1. **Vertex table:** It stores co-ordinate values for each vertex of the object.
 2. **Edge table:** It stores the edge information of polygon.
 3. **Surface table:** It stores the number of surfaces present in the polygon.

► Polygon Surface:

Vertex table

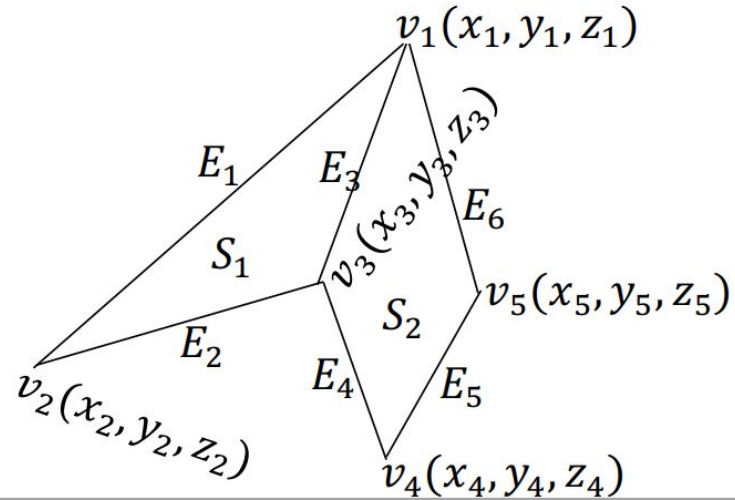
$v_1: x_1, y_1, z_1$
$v_2: x_2, y_2, z_2$
$v_3: x_3, y_3, z_3$
$v_4: x_4, y_4, z_4$
$v_5: x_5, y_5, z_5$

Edge table

$E_1: v_1, v_2$
$E_2: v_2, v_3$
$E_3: v_3, v_1$
$E_4: v_3, v_4$
$E_5: v_4, v_5$
$E_6: v_5, v_1$

Surface table

$S_1: E_1, E_2, E_3$
$S_2: E_3, E_4, E_5, E_6$



The object can be **displayed efficiently** by using data from **tables** and processing them for **surface rendering** and visible surface determination.

► Polygon Surface:

b) Polygon table using forward pointer in edge table

- ❖ **Extra information** can be added to data tables for faster information extraction.
- ❖ **For example:** Edge Table can be expanded to include **forward points** into the polygon table so that **common edges** between the polygon can be identified more rapidly.
- ❖ For above polygon:

Vertex table

$v_1: x_1, y_1, z_1$
$v_2: x_2, y_2, z_2$
$v_3: x_3, y_3, z_3$
$v_4: x_4, y_4, z_4$
$v_5: x_5, y_5, z_5$

Edge table

$E_1: v_1, v_2, S_1$
$E_2: v_2, v_3, S_1$
$E_3: v_3, v_1, S_1, S_2$
$E_4: v_3, v_4, S_2$
$E_5: v_4, v_5, S_2$
$E_6: v_5, v_1, S_2$

Surface table

$S_1: E_1, E_2, E_3$
$S_2: E_3, E_4, E_5, E_6$

► Polygon Surface:

c) Polygon Meshes

- ❖ A polygon mesh is **collection of edges, vertices and polygons** connected such that each edge is shared by **at most two polygons**.
- ❖ An edge connects two vertices and a polygon is a closed sequence of edges. An edge can be **shared by two polygons** and a **vertex** is shared by **at least two edges**.
- ❖ This method can be used to represent a **broad class** of solids/surfaces in graphics.
- ❖ A polygon mesh can be rendered using hidden surface removal algorithms. The polygon mesh can be represented by three ways-
 - Explicit representation
 - Pointers to a vertex list
 - Pointers to an edge list

► Polygon Surface ► Polygon Meshes:

Explicit Representation

- ❖ In Explicit representation, each polygon is represented by a **list of vertex coordinates**.

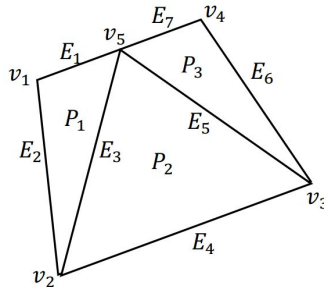
$$P = ((x_1, y_1, z_1), (x_2, y_2, z_2), \dots, (x_n, y_n, z_n))$$

Pointers to a vertex list

- ❖ In Pointers to a vertex list, each vertex is **stored just once**, in vertex list.

$$V = (v_1, v_2, \dots, v_n)$$

- ❖ E.g. A polygon made up of vertices 3, 5, 7, 10 in vertex list be represented as $P_1 = \{3, 5, 7, 10\}$



Representing polygon mesh with each polygon as vertex list.

$$P_1 = \{v_1, v_2, v_5\}$$

$$P_2 = \{v_2, v_3, v_5\}$$

$$P_3 = \{v_3, v_4, v_5\}$$

► Polygon Surface ► Polygon Meshes:

Pointers to an edge list

- ❖ In Pointers to an edge list, we have vertex list V , represent the polygon as a list of pointers to an edge list.
- ❖ Each edge in edge list points to the two vertices in the vertex list. Also to one or two polygon, the edge belongs.
- ❖ Hence, we describe polygon as

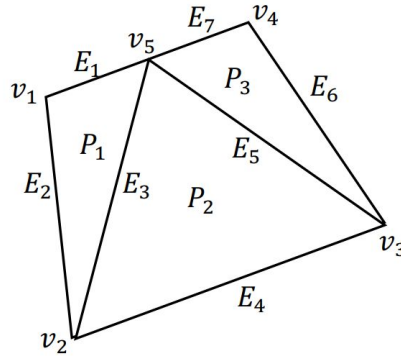
$$P = (E_1, E_2, \dots, E_n) \text{ and}$$

$$\text{an edge as } E = (v_1, v_2, P_1, P_2)$$

Here if edge belongs to only one polygon, either then P_1 or P_2 is null.

► Polygon Surface ► Polygon Meshes:

Pointers to an edge list



For the mesh given above,

$$V = \{v_1, v_2, v_3, v_4, v_5\} = \{(x_1, y_1, z_1), \dots, (x_5, y_5, z_5)\}$$

$$E_1 = (v_1, v_5, P_1, N)$$

$$E_6 = (v_3, v_4, P_3, N)$$

$$E_2 = (v_1, v_2, P_1, N)$$

$$E_7 = (v_4, v_5, P_3, N)$$

$$E_3 = (v_2, v_5, P_1, P_2)$$

$$P_1 = (E_1, E_2, E_3)$$

$$E_4 = (v_2, v_3, P_2, N)$$

$$P_2 = (E_3, E_4, E_5)$$

$$E_5 = (v_3, v_5, P_1, P_3)$$

$$P_3 = (E_5, E_6, E_7)$$

Here, N represents Null.

► Polygon Surface:

d) Plane Equation

- ❖ In this method **polygon surface is represented by the equation** of plane in the coordinate system. The 3D object is represented through the set of equations. The equation for plane surface can be expressed as

$$Ax + By + Cz + D = 0$$

- ❖ Where x, y, z is any point on the plane and A, B, C & D are coefficient of plane equation and represents the spatial orientation of the polygon surface in space coordinate system. Hence, the value of coefficient must be known to represent the 3D object.
- ❖ The value of A, B, C & D can be obtained by **solving a set of three plane equation** using coordinate of three non-collinear point on plane.

► Polygon Surface:

d) Plane Equation

❖ Let us assume that three vertices of plane are (x_1, y_1, z_1) , (x_2, y_2, z_2) , (x_3, y_3, z_3) . Then,

$$Ax_1 + By_1 + Cz_1 + D = 0$$

$$Ax_2 + By_2 + Cz_2 + D = 0$$

$$Ax_3 + By_3 + Cz_3 + D = 0$$

By Cramer's rule

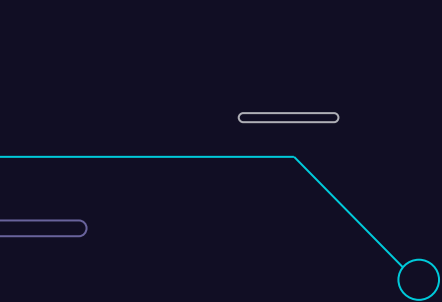
$$A = \begin{vmatrix} 1 & y_1 & z_1 \\ 1 & y_2 & z_2 \\ 1 & y_3 & z_3 \end{vmatrix} \quad B = \begin{vmatrix} x_1 & 1 & z_1 \\ x_2 & 1 & z_2 \\ x_3 & 1 & z_3 \end{vmatrix} \quad C = \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix} \quad D = \begin{vmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{vmatrix}$$

For any points (x, y, z)

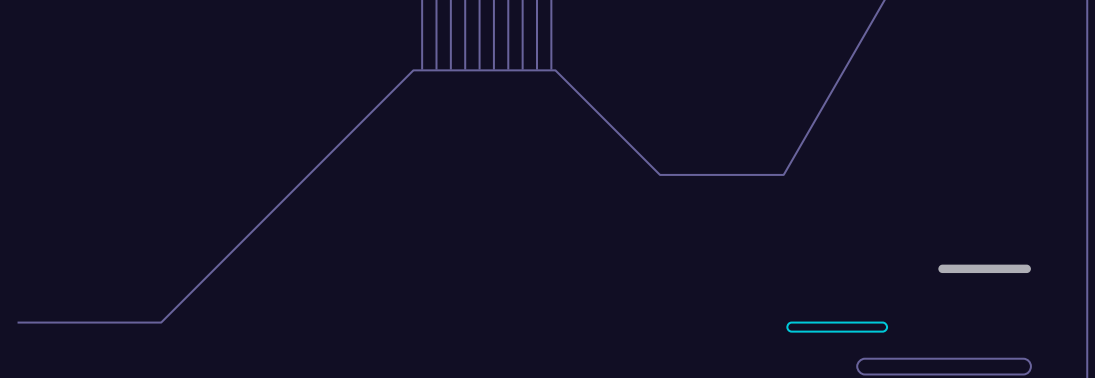
If $Ax + By + Cz + D \neq 0$, then (x, y, z) is not on the plane.

If $Ax + By + Cz + D < 0$, then (x, y, z) is inside the plane i. e. invisible side

If $Ax + By + Cz + D > 0$, then (x, y, z) is lies outside the surface.



► Wireframe Representation

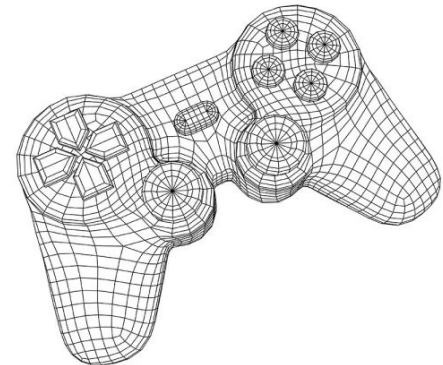
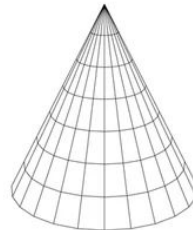
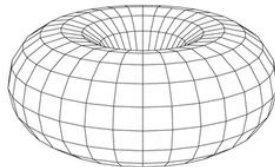
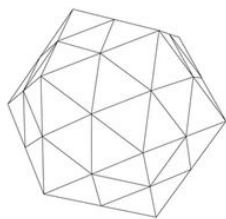


5.1.2

► Boundary Representation:

2. Wireframe representation:

- ❖ If an object may be represented through the **collection of points and line** then it's called wireframe object.
- ❖ It's used to define **skeleton** of 3D objects in terms of points and lines.
- ❖ Mostly use in field of **engineering drawings** to represent the **drawing of structure** and missing parts.
- ❖ Wireframe represented object requires **less memory for storage and fast for display** and surface rendering.
- ❖ In this method the scenes represented are **not realistic.**





► **Blobby Objects**

5.1.3

► Boundary Representation:

3. Blobby Objects:

- ❖ Object that **don't maintain a fixed shape** but changes their surface characteristics during **motion or closer to another object** are called blobby object.
- ❖ **Example:** molecular structure, water droplet, muscle shape in human body etc.
- ❖ Most common function used for blobby object is **Gaussian density function**. A surface function is defined as:

$$f(x, y, z) = \sum_k b_k e^{-a_k r_k^2} - T = 0$$

Where, $r_k^2 = \sqrt{x_k^2 + y_k^2 + z_k^2}$

T= some specified threshold

a, b= parameters used to adjust the amount of blobbiness for individual object.

► Bloby Objects:



(a)



(b)

Figure: Bloby Muscle in Human

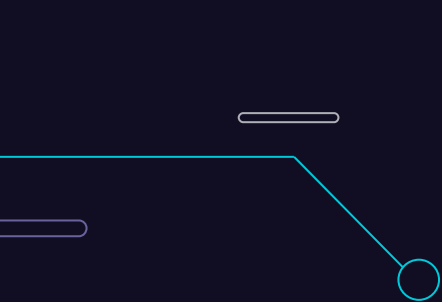
► Bloby Objects:

❖ Advantages

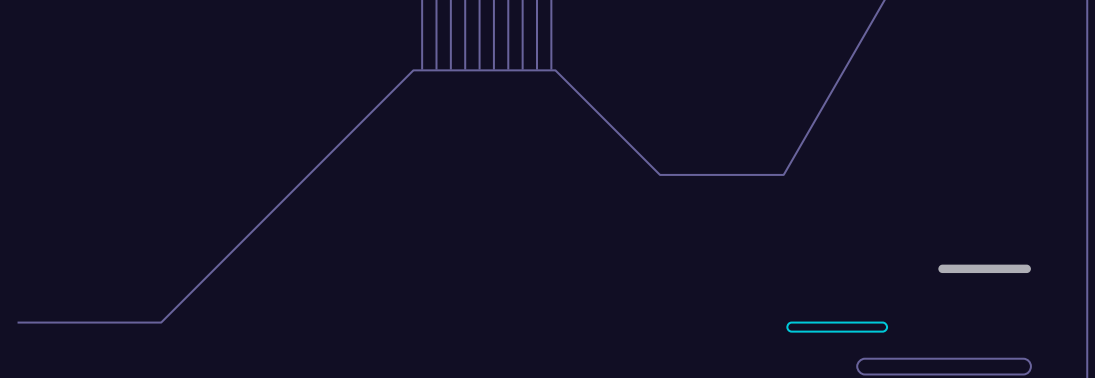
- Can represent **organic, blobby or liquid line** structures.
- Suitable for **modeling natural phenomena** like water, human body.
- Surface properties can be easily derived from mathematical equations.

❖ Disadvantages

- Requires expensive computation
- Requires special rendering engine
- Not supported by most graphics hardware



► Representing Curves



5.2

► Curve:

- ❖ A curve is an infinitely large set of points. **Each point has two neighbours** except endpoints. A curve is defined as a smoothly- flowing continuous line that has bent.

- ❖ **Types of Curves:**



1. **Implicit Curve** : It defines the set of points on a curve by employing a procedure that can test to see **if a point is on the curve**. It is defined by an implicit function as:
 - $f(x,y)=0$ In 2D Space, Example: $x^2+y^2-r^2=0$
 - $f(x,y,z)=0$ in 3-D Space
2. **Explicit Curve**: The explicit representation is not general, since it cannot represent vertical lines and is **single valued**. For each x , a single y value is computed. The function **$y = f(x)$** is the explicit representation of curve.
3. **Parametric Curve**: Curves having parameter is called as parametric curves.

► Parametric Cubic Curve:

❖ Parametric Curves:

- ❖ Curves having parameter is called as parametric curves.
- ❖ The explicit and implicit curves can only be used **when the function is known**.
- ❖ A 2D parametric curve has the following form:

$$P(t) = f(t), g(t)$$

- ❖ The functions f and g become the (x,y) **coordinates of any point on the curve** and the points are obtained when the **parameter t** is varied over a certain interval $[a,b]$, normally $[0,1]$.

❖ Parametric Cubic Curves:

- Higher the degree of the curve, we will have **many parameters** that we can set to form the desired shape, but the evaluation of points will be costly.
- Lower the degree of curve, we may **not have enough parameters** which which we work.

► Parametric Cubic Curve:

◆ Algebraic Representation of Parametric Cubic Curves:

- All of these curves are just parametric algebraic polynomials expressed in different bases (P_x, P_y, P_z).
- **parameter u** is varied over a certain interval $[a, b]$, normally $[0, 1]$

➤ Parametric Linear Curve:

■ $p(u) = au + b$

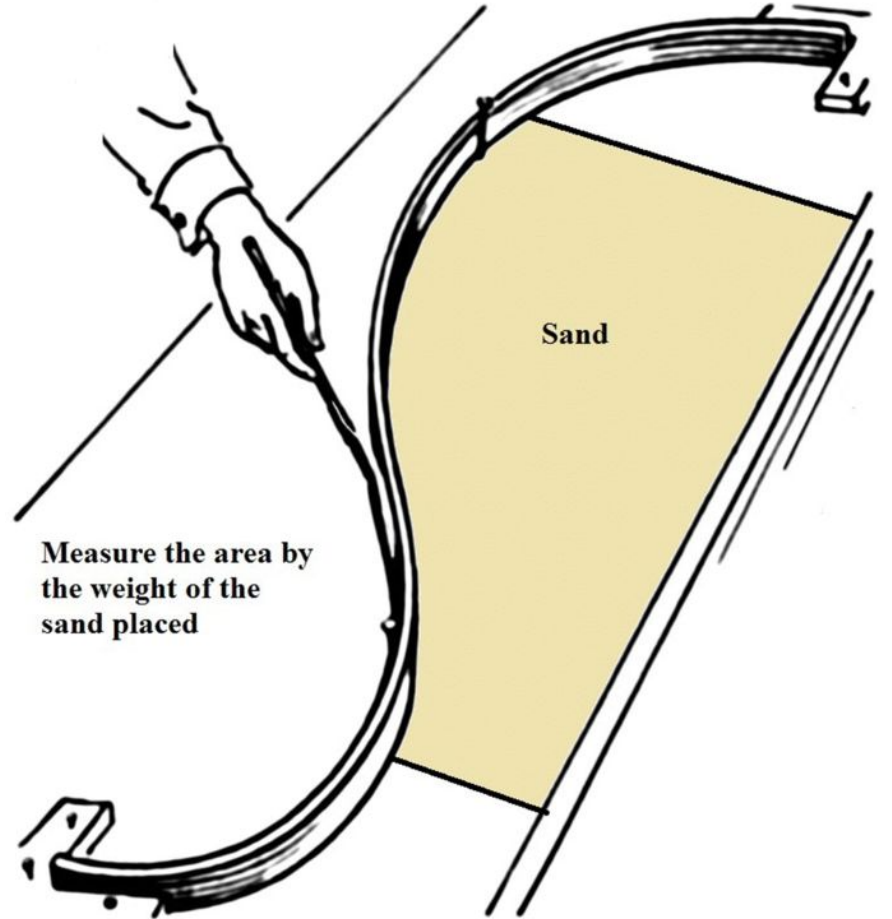
- $x = a_x u + b_x$
- $y = a_y u + b_y$
- $z = a_z u + b_z$

➤ Parametric Cubic Curve:

■ $p(u) = au^3 + bu^2 + cu + d$

- $x = a_x u^3 + b_x u^2 + c_x u + d$
- $y = a_y u^3 + b_y u^2 + c_y u + d$
- $z = a_z u^3 + b_z u^2 + c_z u + d$

► Spline Curve:



► Representing Curve:

5. Spline Representation:

- ❖ A Spline is a **flexible strips used to produce smooth curve** through a designated set of points. A curve drawn with these set of points is spline curve. Spline curves are used to model 3D object surface shape smoothly.
- ❖ A spline curve is a mathematical representation that **allow the user to design and control shape of complex curve** and surface.
- ❖ Here, user enters a sequence of points called **control points (for shape control)** & a curve is constructed whose **shape closely follows these control points.**
- ❖ **If Control Points = n , Equation of Curve = x^{n-1}**
- ❖ **Two types** of spline curve:
 1. **Interpolating Curve**
 2. **Approximating Curve**

► Spline Curve:

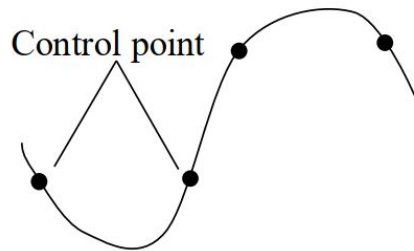
Two types of spline curve:

❖ Interpolating curve:

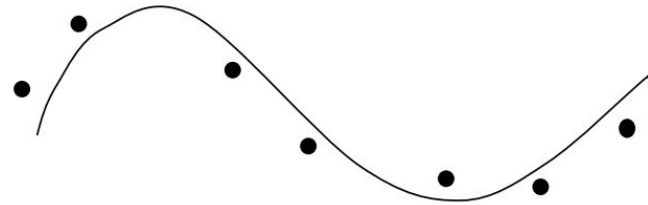
- A curve that **actually passes through each control point**.
- Interpolation curves are commonly used to digitize drawing or to specify animation paths.

❖ Approximating curve:

- A curve that passes **near to control point** but **not necessary through them**.
- Approximation curves are used as design tools to structure object surfaces.



Interpolating curve



Approximating curve

► Spline Curve:

- ❖ Spline curve also **used for design of** automobile bodies, spacecraft, specification of animation path, home appliance etc.
- ❖ The **three degree polynomial** known as **cubic polynomial** is typically used for constructing smooth curve in computer graphics because of following reasons:
 - It is **lowest degree** polynomial that can support an **inflection** (a point at which curve crosses its tangent i.e. curve changes from concave to convex).
 - The curves are smooth like this  and not jumpy like this 

► Spline Specifications:

There are three equivalent methods for specifying a particular spline representation:

- a) **Boundary condition:** We can state the set of boundary conditions that are imposed on the spline.

$$x(u) = a_x u^3 + b_x u^2 + c_x u + d_x \quad 0 \leq u \leq 1$$

Boundary condition for this curve can be set for $x(0)$, $x(1)$, $x'(0)$ & $x'(1)$. These four conditions are sufficient to determine the values of four coefficient a_x , b_x , c_x & d_x .

- b) **Characterizing matrix:** We can state the matrix that characterizes the spline.

From the boundary condition, the characterizing matrix for spline is:

$$x(u) = [u^3 \quad u^2 \quad u \quad 1] \begin{bmatrix} a_x \\ b_x \\ c_x \\ d_x \end{bmatrix}$$
$$= U \cdot C$$

- c) **Blending Function:** We can state the set of blending functions (or basis functions) that determine how specified geometric constraints on the curve are combined to calculate positions along the curve path.

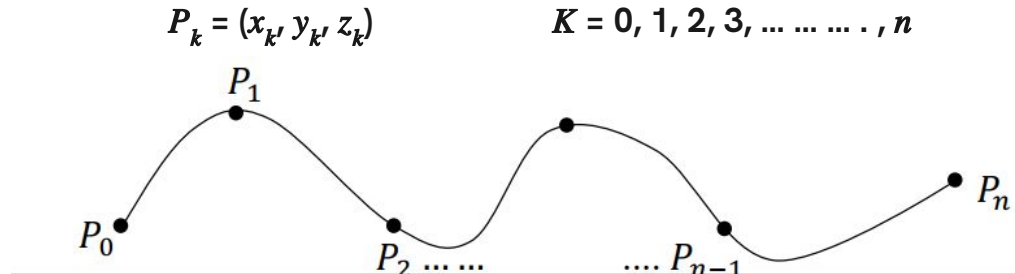
$$x(u) = \sum_{k=0}^3 g_k \cdot BF_k(u)$$

g_k = Geometric constrain parameter

$BF_k(u)$ = Polynomial blending function

► Cubic Spline:

- ❖ It is used to **set up path for object motions** or to provide a **representation for an existing object** or drawings.
- ❖ Compared to higher-order polynomials, cubic splines requires **less calculation and memory** and **they are more stable**. Compared to lower-order polynomials, cubic splines are **more flexible** for modeling arbitrary curve shapes.
- ❖ Cubic interpolation spline is obtained by fitting the **input points** with a piecewise cubic polynomial **curve** that passes through every **control points**.
- ❖ Suppose we have $n+1$ control points having coordinates



► Cubic Spline:

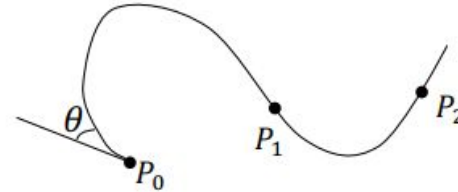
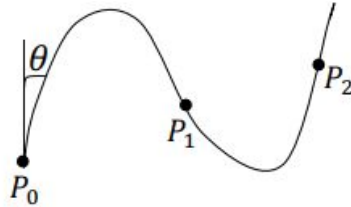
- ❖ A parametric cubic polynomial that is to be fitted between each pair of control points have following equations:

$$\begin{aligned}x(u) &= a_x u^3 + b_x u^2 + c_x u + d_x \\y(u) &= a_y u^3 + b_y u^2 + c_y u + d_y \\z(u) &= a_z u^3 + b_z u^2 + c_z u + d_z\end{aligned} \quad (0 \leq u \leq 1)$$

- ❖ We **need to determine the values** of the four coefficients a , b , c , and d in the polynomial representation for each of the n curve section.
- ❖ We do this by **setting enough boundary conditions** at the “joints” between curve sections we can obtain numerical values for all the coefficients.
 - Cubic splines are more flexible for modeling arbitrary curve shapes.

► Hermite Interpolation (Hermite Curve):

- ❖ They are used to **smoothly interpolate** between key-points. (like object movement in animation or camera control)
- ❖ It is an **interpolating piecewise cubic polynomial** with a specified tangent at each control point.



If we change the control point at P_0 , then the curve will also change, so that angle θ between P_0 and tangent at P_0 will remain constant.

- It has local control over the curve i.e. each curve section depend on its end point only.
- The vector equivalence of Hermite curve is

$$P(u) = au^3 + bu^2 + cu + d \dots\dots\dots (i)$$

Where, x component of $P(u)$ is

$$x(u) = a_x u^3 + b_x u^2 + c_x u + d_x$$

Similarly, y and z component

$$y(u) = a_y u^3 + b_y u^2 + c_y u + d_y$$

$$z(u) = a_z u^3 + b_z u^2 + c_z u + d_z$$

► Hermite Interpolation (Hermite Curve):

❖ To calculate a **Hermite curve**, you need the following **four vectors**:

- **P1** : the start point of the curve (**Position Vector**) $P1' = T1$
- **T1**: the tangent (e.g., direction and speed) at the start point
- **P2**: the endpoint of the curve
- **T2**: the tangent (e.g., direction and speed) at the endpoint

❖ **Matric Form of Hermite Curve:**

- **Vector U** : The interpolation-point and its powers up to 3 (u values)
- **Vector C** : The parameters of our Hermite curve
- **Matrix h**: The matrix form of the 4 Hermite polynomials

To calculate a point on the curve, we first build the Vector U, multiply it with cubic Hermite basis matrix h and then multiply with C, to get:

$$P = U * h * C$$

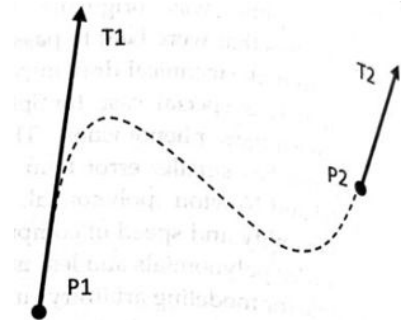


Figure 6.8: Hermite Curve

► Hermite Interpolation (Hermite Curve):

The **standard cubic Hermite basis matrix**, **h** is:

$$H = \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

Point on the Curve: **P = U*h*C**, becomes:

$$P = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} P_1 \\ P_2 \\ T_1 \\ T_2 \end{bmatrix}$$

After multiplication, this gives the **cubic Hermite interpolation polynomial**:

$$P(u) = (2u^3 - 3u^2 + 1)P_1 + (-2u^3 + 3u^2)P_2 + (u^3 - 2u^2 + u)T_1 + (u^3 - u^2)T_2$$

► Representing Curve:

6. Bezier Curve:

- ❖ Bezier curve is developed by the French engineer **Pierre Bezier** for the design of **Renault automobile bodies**.
- ❖ The Bezier curve is a **parametric curve $P(t)$** that is a polynomial function of the parameter t . The **degree of the polynomial depends upon the number of points used to define the curve.**
 - It is an **approximating spline (approximate spline curve)** widely used in various CAD system.
 - Bezier curve is generated under the **control of points** known as **control points**.
- ❖ General Bezier curve for **$(n+1)$ control point**, denoted as $p_k = (x_k, y_k, z_k)$ with 'k' varying from 0 to n is given by

$$P(u) = \sum_{k=0}^n p_k BEZ_{k,n}(u), \quad 0 \leq u \leq 1 \quad \dots \dots \dots (i)$$

► Bezier Curve:

Where, $P(u)$ is a point on Bezier Curve.

p_k is a control point.

$BEZ_{k,n}(u)$ is a Bezier blending function also known as *Bernstein Polynomial*.

Bezier blending function is defined as

$$BEZ_{k,n}(u) = C(n, k)u^k(1 - u)^{n-k}$$

Where, $C(n, k) = \frac{n!}{k!(n-k)!}$

Individual x, y, z coordinates an a Bezier curve is given by,

$$x(u) = \sum_{k=0}^n x_k BEZ_{k,n}(u)$$

$$y(u) = \sum_{k=0}^n y_k BEZ_{k,n}(u)$$

$$z(u) = \sum_{k=0}^n z_k BEZ_{k,n}(u)$$

► Bezier Curve:

In general , a Bezier Curve is a polynomial of degree that is **one less** than the designated number of control points:

- ❖ **Three points** generate a **parabola**
- ❖ **Four points** generate a **cubic curve**
- ❖ i.e ($n-1$) is the degree of polynomial for the n control points

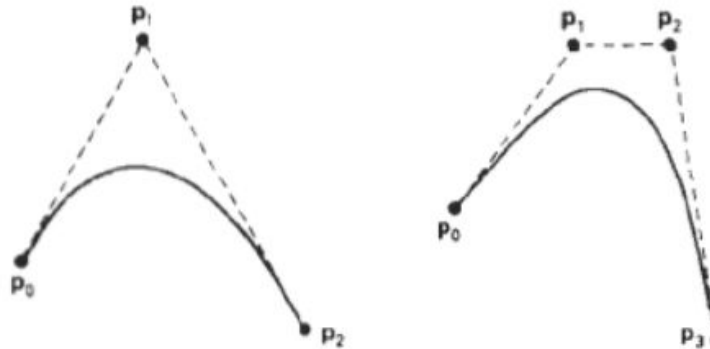


Fig: Bezier curve generated by three and four control point.

► Bezier Curve:

Properties of Bezier curve:

- a) The basis functions are real.
- b) The Bezier curve always passes through first and last control point i.e. $p(0) = p_0$ & $p(1) = p_n$.
- c) The degree of polynomial representing Bezier curve is one less than the number of control points.
- d) The Bezier curve always follows convex hull formed by control points.
- e) The Bezier curve always lies inside the polygon formed by control points.
- f) Bezier blending functions are positive and sum is equal to 1.
$$\sum_{k=0}^n BEZ_{k,n}(u) = 1$$
- g) The direction of the tangent vector at the end points is same like vector determined by first and last segment.

► Cubic Bezier Curve:

- It is a Bezier curve generated by four control points.
- General equation for cubic Bezier curve is

$$P(u) = \sum_{k=0}^3 p_k BEZ_{k,n}(u), \quad 0 \leq u \leq 1 \quad \dots \dots \dots (i)$$

$$P(u) = p_0 BEZ_{0,3}(u) + p_1 BEZ_{1,3}(u) + p_2 BEZ_{2,3}(u) + p_3 BEZ_{3,3}(u)$$

Where,

$$BEZ_{0,3}(u) = C(3,0)u^0(1-u)^{3-0} = \frac{3!}{0!(3-0)!} \times (1-u)^3 = (1-u)^3$$

Similarly,

$$BEZ_{1,3}(u) = 3u(1-u)^2$$

$$BEZ_{2,3}(u) = 3u^2(1-u)$$

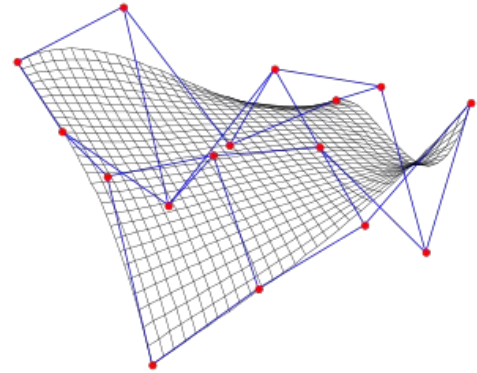
$$BEZ_{3,3}(u) = u^3$$

$$\therefore P(u) = p_0(1-u)^3 + p_1 3u(1-u)^2 + p_2 3u^2(1-u) + p_3 u^3$$

► Bezier Surface:

- ❖ Generalizations of **Bezier curves to higher dimensions** are called Bezier surfaces.
- ❖ The parametric vector function for the Bezier surface is formed as the **Cartesian product** of **Bezier blending function**:

$$P(u, v) = \sum_{j=0}^m \sum_{k=0}^n p_{j,k} BEZ_{j,m}(v) BEZ_{k,n}(u)$$



- ❖ With $p_{j,k}$ specifying the location of the $(m+1)$ by $(n+1)$ control points.
- ❖ Bezier surfaces have the same properties as Bezier curves, and they provide a convenient method for interactive design applications.

► Bezier Surface:

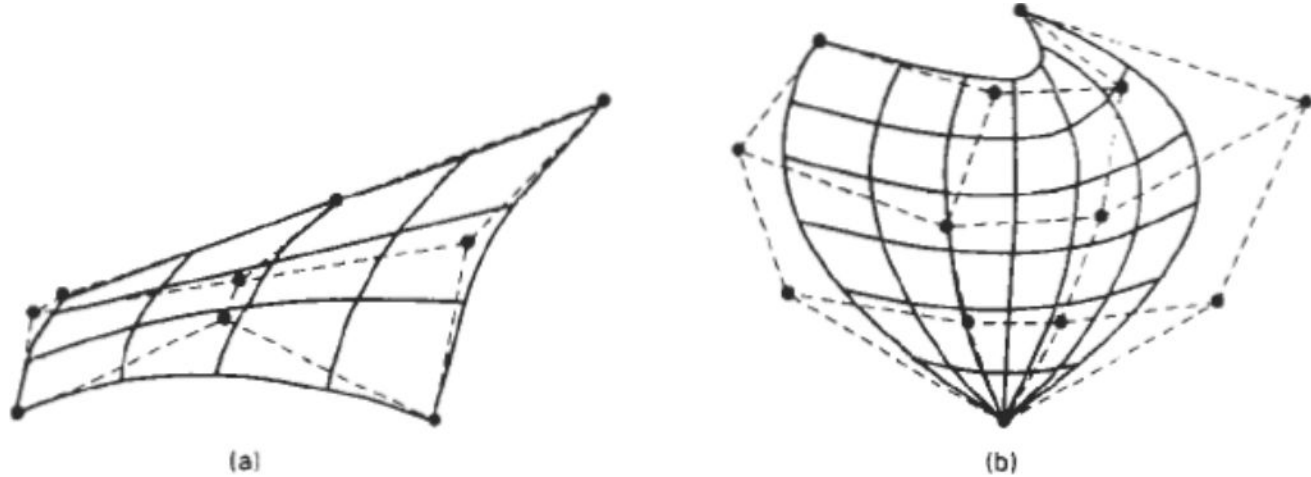


Fig: Bezier surfaces constructed for (a) $m = 3$, $n = 3$, and (b) $m = 4$, $n = 4$. Dashed lines connect the control points

Q. Construct Bezier curve for control points (4, 2), (8, 8) and (16, 4).

Solution:

Given control points

$$p_0 = (x_0, y_0) = (4, 2)$$

$$p_1 = (x_1, y_1) = (8, 8)$$

$$p_2 = (x_2, y_2) = (16, 4)$$

Here, degree (or order) $n = 2$

We have basis function as

$$P(u) = \sum_{k=0}^n p_k BEZ_{k,n}(u), \quad 0 \leq u \leq 1$$

$$= \sum_{k=0}^2 p_k BEZ_{k,2}(u), \quad 0 \leq u \leq 1$$

$$\therefore P(u) = p_0 BEZ_{0,2}(u) + p_1 BEZ_{1,2}(u) + p_2 BEZ_{2,2}(u)$$

Parametric equations are;

$$x(u) = x_0 BEZ_{0,2}(u) + x_1 BEZ_{1,2}(u) + x_2 BEZ_{2,2}(u) \dots \dots \dots (i)$$

$$y(u) = y_0 BEZ_{0,2}(u) + y_1 BEZ_{1,2}(u) + y_2 BEZ_{2,2}(u) \dots \dots \dots (ii)$$

Now,

$$BEZ_{0,2}(u) = C(2,0)u^0(1-u)^{2-0} = \frac{2!}{0!(2-0)!} \times (1-u)^2 = (1-u)^2$$

$$BEZ_{1,2}(u) = C(2,1)u^1(1-u)^{2-1} = \frac{2!}{1!(2-1)!} \times u(1-u)^1 = 2u(1-u)$$

$$BEZ_{2,2}(u) = C(2,2)u^2(1-u)^{2-2} = \frac{2!}{2!(2-2)!} \times u^2(1-u)^0 = u^2$$

Putting these values in eq. (i) & (ii) we get;

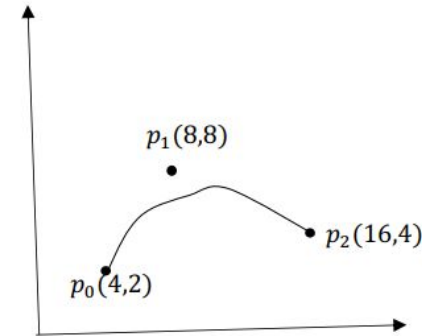
$$x(u) = x_0(1-u)^2 + x_1 2u(1-u) + x_2 u^2 = 4u^2 + 8u + 4$$

$$y(u) = y_0(1-u)^2 + y_1 2u(1-u) + y_2 u^2 = -10u^2 + 12u + 2$$

Now,

	$x(u)$	$y(u)$
$u=0$	4	2
$u=0.2$	5.76	4.0
$u=0.4$	7.84	5.20
$u=0.6$	10.24	5.6
$u=0.8$	12.96	5.2
$u=1$	16	4

Drawing these points we get:



Q. Construct the Bezier curve of order 3 with 4 vertices of the control polygon $p_0(0, 0)$, $p_1(1, 2)$, $p_2(3, 2)$ & $p_3(2, 0)$. Generate at least 5 points on the curve.

Solution:

Given control points

$$p_0 = (x_0, y_0) = (0, 0)$$

$$p_1 = (x_1, y_1) = (1, 2)$$

$$p_2 = (x_2, y_2) = (3, 2)$$

$$p_3 = (x_3, y_3) = (2, 0)$$

Here, degree $n=3$

We have basis function as

$$P(u) = \sum_{k=0}^n p_k BEZ_{k,n}(u), \quad 0 \leq u \leq 1$$

$$= \sum_{k=0}^3 p_k BEZ_{k,3}(u), \quad 0 \leq u \leq 1$$

$$\therefore P(u) = p_0 BEZ_{0,3}(u) + p_1 BEZ_{1,3}(u) + p_2 BEZ_{2,3}(u) + p_3 BEZ_{3,3}(u)$$

Parametric equations are;

$$x(u) = x_0 BEZ_{0,3}(u) + x_1 BEZ_{1,3}(u) + x_2 BEZ_{2,3}(u) + x_3 BEZ_{3,3}(u) \dots \dots \dots (i)$$

$$y(u) = y_0 BEZ_{0,3}(u) + y_1 BEZ_{1,3}(u) + y_2 BEZ_{2,3}(u) + y_3 BEZ_{3,3}(u) \dots \dots \dots (ii)$$

Now,

$$BEZ_{0,3}(u) = C(3,0)u^0(1-u)^{3-0} = \frac{3!}{0!(3-0)!} \times (1-u)^3 = (1-u)^3$$

$$BEZ_{1,3}(u) = C(3,1)u^1(1-u)^{3-1} = \frac{3!}{1!(3-1)!} \times u(1-u)^2 = 3u(1-u)^2$$

$$BEZ_{2,3}(u) = C(3,2)u^2(1-u)^{3-2} = \frac{3!}{2!(3-2)!} \times u^2(1-u)^1 = 3u^2(1-u)$$

$$BEZ_{3,3}(u) = C(3,3)u^3(1-u)^{3-3} = \frac{3!}{3!(3-3)!} \times u^3(1-u)^0 = u^3$$

Putting these values in eq. (i) & (ii) we get;

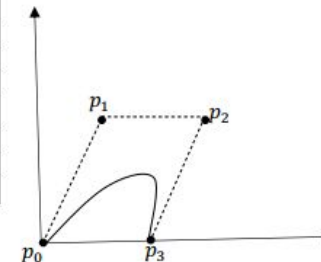
$$x(u) = x_0(1-u)^3 + x_1 3u(1-u)^2 + x_2 3u^2(1-u) + x_3 u^3 = -4u^3 + 3u^2 + 3u$$

$$y(u) = y_0(1-u)^3 + y_1 3u(1-u)^2 + y_2 3u^2(1-u) + y_3 u^3 = -6u^2 + 6u$$

Now,

	$x(u)$	$y(u)$
$u=0$	0	0
$u=0.15$	0.50	0.76
$u=0.35$	1.24	1.36
$u=0.5$	1.75	1.50
$u=0.65$	2.12	1.36
$u=0.85$	2.14	0.76
$u=1$	2	0

Plotting the graph;



► Representing Curve:

7. B-Spline Curve:

- ❖ B-spline curve is a set of **piecewise polynomial segments** that passes **close** to a set of control points.
- ❖ It has two advantage over Bezier curve:
 - The degree of B-spline polynomial can be **set independently** of the number of control points.
 - It allows **local control** over the shape of a spline curve.
- ❖ General equation of B-spline curve is given by

$$P(u) = \sum_{k=0}^n p_k B_{k,d}(u), \quad 0 \leq u \leq n + d, \quad 2 \leq d \leq n + 1$$

► B-Spline Curve:

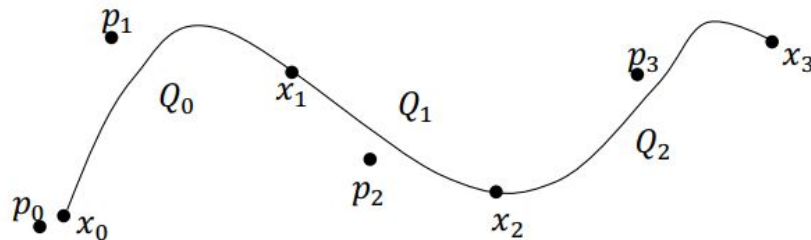
Where, p_k is a set of $(n+1)$ control points.

$B_{k,d}(u)$ is the B-spline blending function.

Blending function for B-spline curves are defined by

$$B_{k,1}(u) = \begin{cases} 1, & \text{if } u_k \leq u < u_{k+1} \\ 0, & \text{Otherwise} \end{cases}$$

$$B_{k,d}(u) = \frac{u - u_k}{u_{k+d-1} - u_k} B_{k,d-1}(u) + \frac{u_{k+d} - u}{u_{k+d} - u_{k+1}} B_{k+1,d-1}(u)$$



$p_0, p_1, p_2, p_3 \rightarrow$ Control point

$x_0, x_1, x_2, x_3 \rightarrow$ Knot values

$Q_0, Q_1, Q_2 \rightarrow$ Curve segment

- The knots produce a vector that defines the domain of the curve.

► Properties of B-Spline Curve:

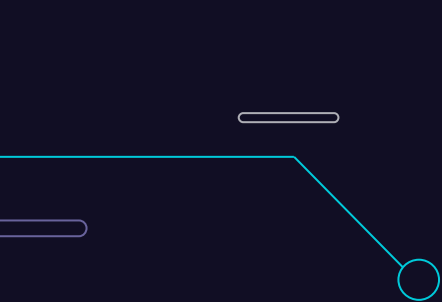
- a) The polynomial curve has $d - 1$ degree.
- b) For $(n+1)$ control points, the curve is described with $(n+1)$ blending function.
- c) Each blending function $B_{k,d}$ is defined over 'd' sub-interval of the total range of 'u', starting at knot value u_k .
- d) The sum of B-spline basis functions for any parameter value is 1

$$\sum_{k=0}^n B_{k,d}(u) = 1$$

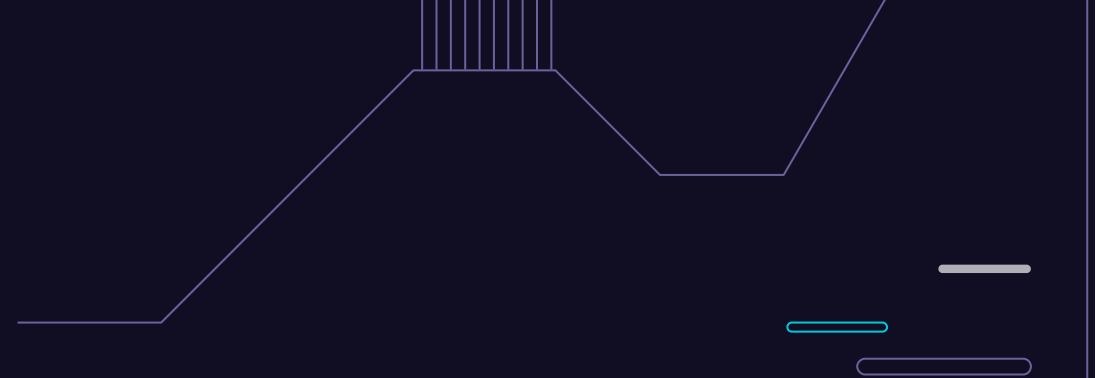
- e) Each basis function is positive or zero for all parameter value.
- f) The range of parameter 'u' is divided into $(n+d)$ sub interval by $(n+d+1)$ values specified in knot vector.
- g) Each section of spline curve is influenced by 'd' control point.
- h) Any one control point can affect the shape of at most 'd' curve sections.
- i) The maximum order of curve is equal to the number of vertices of defining polygon.
- j) The curve generally follows the shape of defining polygon.
- k) The degree of B-spline polynomial is independent on the number of vertices of defining polygon.

► B-Spline Surface:

- ❖ On the basis of the knot points and interval length of segment there are two types of spline;
 - **Periodic B-spline:**
 - Knot points are **equi-space** to each other and splines are generated through the set of the **equi-interval segments** then such splines are called periodic splines.
 - **Aperiodic B-spline:**
 - If knot points are **not equi-space** to each other and splines are not generated through the set of the equi interval segments then such splines are called aperiodic B-splines.



► Quadric Surface : Sphere & Ellipsoid



5.3

► Boundary Representation:

8. Quadratic Surface:

- ❖ Quadric Surface is one of the frequently used 3D objects surface representation. The quadric surface can be represented by a second degree polynomial. This includes:

► Sphere:

- For a set of surface point (x, y, z) spherical surface is represented by equation,

$$(x - x_c)^2 + (y - y_c)^2 + (z - z_c)^2 = r^2$$

Where, (x_c, y_c, z_c) is the center of sphere and r is the radius of the sphere.

► Ellipsoid:

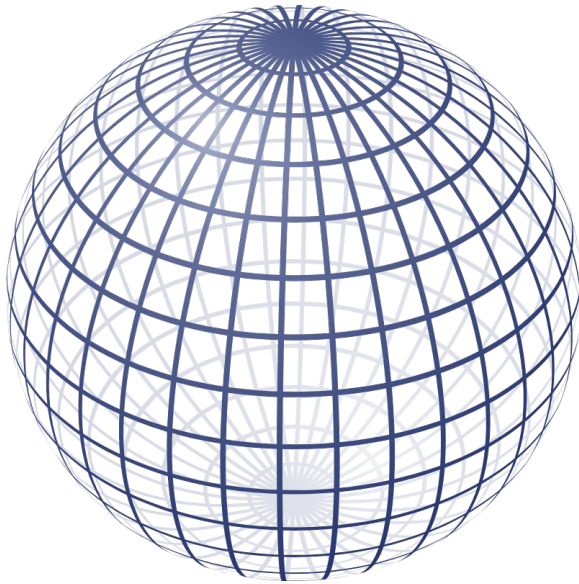
- If (x, y, z) be the any point on ellipsoid with the radius a, b & c along x, y & z axis then represented as;

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} + \frac{z^2}{c^2} = 1$$

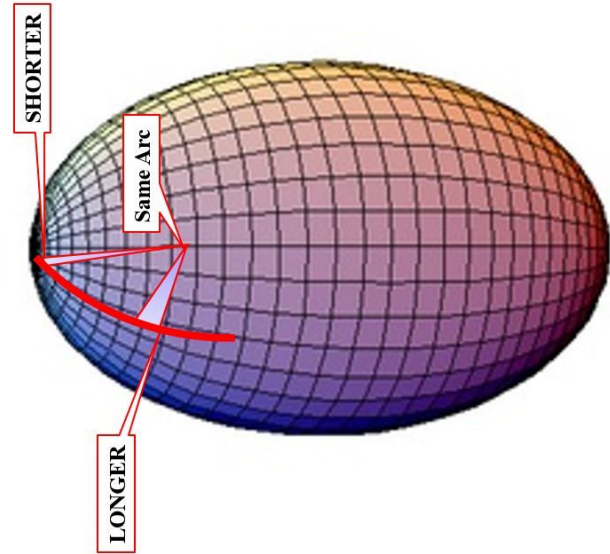
► Boundary Representation:

8. Quadratic Surface:

► Sphere:



Ellipsoid:



► Assignment - 5:

Due Date : 8th February, 2026

- ❖ Explain polygon surface representation using Polygon table and polygon meshes. 2080
- ❖ What is polygon table? Explain the use of this method in 3D object representation. 2079
- ❖ What are blobby objects? How it is represented? Explain the wireframe representation of 3D objects. 2079
- ❖ Explain about parametric curve. Describe the properties of Bezier curve. 2079
- ❖ Define fractal. Explain the Bezier curve and B-Spline curve. 2078
- ❖ How polygon table is used in representing polygons? Explain the representations of any three curves. 2077
- ❖ What is the purpose of wireframe representation? Describe about boundary and space partitioning.
- ❖ Given a bezier curve with 4 control points- $B_0[1\ 0]$, $B_1[3\ 3]$, $B_2[6\ 3]$, $B_3[8\ 1]$ Determine any 5 points lying on the curve. Also, draw a rough sketch of the curve.



▶ **THANKS!**

Do you have any questions?

theciceerguy@p_m.me

9841893035

ghimires.com.np

